

Глава 4

Типы данных СУБД PostgreSQL

После первоначального знакомства с языком SQL имеет смысл немного упорядочить полученные вами знания. Речь идет о типах данных, применяемых в СУБД PostgreSQL. Вообще, типы данных — это одно из базовых понятий любого языка программирования, и язык SQL в этом плане не является исключением.

PostgreSQL имеет очень разнообразный набор встроенных типов данных, т. е. тех типов, которые СУБД предоставляет в распоряжение пользователя, как говорят, по умолчанию. Мы намеренно употребили здесь термин «встроенные», поскольку пользователь имеет возможность создавать и свои собственные типы данных, которые затем можно включить в систему и использовать их так же, как и встроенные. Такая возможность адаптации системы типов данных к конкретным ситуациям является одной из отличительных черт PostgreSQL.

В этой главе мы расскажем лишь о самых основных типах данных, поскольку в вашем распоряжении всегда имеется полная документация. Настоящее учебное пособие не является ее заменой, оно призвано лишь помочь вам сделать первые шаги в освоении богатого мира типов данных PostgreSQL.

Типы данных объединены в группы, в рамках этих групп они имеют некоторые общие свойства, но также они имеют и различия.

4.1. Числовые типы

Группа числовых типов данных включает в себя целый ряд разновидностей: целочисленные типы, числа фиксированной точности, типы данных с плавающей точкой, последовательные типы (*serial*).

В составе целочисленных типов находятся следующие представители: *smallint*, *integer*, *bigint*. Если атрибут таблицы имеет один из этих типов, то он позволяет хранить только целочисленные данные. При этом перечисленные типы различаются по количеству байтов, выделяемых для хранения данных. В PostgreSQL существуют псевдонимы для этих стандартизированных имен типов, а именно: *int2*, *int4* и *int8*. Число байтов отражается в имени типа.

При выборе конкретного целочисленного типа принимают во внимание диапазон допустимых значений и затраты памяти. Зачастую тип `integer` считается оптимальным выбором с точки зрения достижения компромисса между этими показателями.

Числа фиксированной точности представлены двумя типами — `numeric` и `decimal`. Однако они являются идентичными по своим возможностям. Поэтому мы будем проводить изложение на примере типа `numeric`. Для задания значения этого типа используются два базовых понятия: масштаб (`scale`) и точность (`precision`). Масштаб показывает число значащих цифр, стоящих справа от десятичной точки (запятой). Точность указывает общее число цифр как до десятичной точки, так и после нее. Например, у числа 12.3456 точность составляет 6 цифр, а масштаб — 4 цифры.

Параметры этого типа данных указываются в круглых скобках после имени типа: `numeric(точность, масштаб)`. Например, `numeric(6, 2)`.

Его главное достоинство — это обеспечение *точных* результатов при выполнении вычислений, когда это, конечно, возможно в принципе. Это оказывается возможным при выполнении сложения, вычитания и умножения. Числа типа `numeric` могут хранить очень большое количество цифр: 131 072 цифры — до десятичной точки (запятой), 16 383 — после точки. Однако нужно учитывать, что такая точность достигается за счет замедления вычислений по сравнению с целочисленными типами и типами с плавающей точкой. При этом для хранения числа затрачивается больше памяти, чем в случае целых чисел.

Данный тип следует выбирать для хранения денежных сумм, а также в других случаях, когда требуется гарантировать точность вычислений.

Представителями типов данных с плавающей точкой являются типы `real` и `double precision`. Они представляют собой реализацию стандарта IEEE «Standard 754 for Binary Floating-Point Arithmetic». Тип данных `real` может представить числа в диапазоне, как минимум, от $1E-37$ до $1E+37$ с точностью не меньше 6 десятичных цифр. Тип `double precision` имеет диапазон значений примерно от $1E-307$ до $1E+308$ с точностью не меньше 15 десятичных цифр.

При попытке записать в такой столбец слишком большое или слишком маленькое значение будет генерироваться ошибка. Если точность вводимого числа выше допустимой, то будет иметь место округление значения. А вот при вводе очень маленьких чисел, которые невозможно представить значениями, отличными от нуля, будет генерироваться ошибка потери значимости, или исчезновения значащих разрядов (an underflow error).

При работе с числами таких типов нужно помнить, что сравнение двух чисел с плавающей точкой на предмет равенства их значений может привести к неожиданным результатам.

Например:

```
SELECT 0.1::real * 10 = 1.0::real;
```

```
?column?
-----
f
(1 строка)
```

В дополнение к обычным числам эти типы данных поддерживают и специальные значения Infinity (бесконечность), $-\text{Infinity}$ (отрицательная бесконечность) и NaN (не число).

PostgreSQL поддерживает также тип данных `float`, определенный в стандарте SQL. В объявлении типа может использоваться параметр: `float(p)`. Если его значение лежит в диапазоне от 1 до 24, то это будет равносильно использованию типа `real`, а если же значение лежит в диапазоне от 25 до 53, то это будет равносильно использованию типа `double precision`. Если же при объявлении типа параметр не используется, то это также будет равносильно использованию типа `double precision`.

Последним из числовых типов является тип `serial`. Однако он фактически реализован не как настоящий тип, а просто как удобная замена целой группы SQL-команд. Тип `serial` удобен в тех случаях, когда требуется в какой-либо столбец вставлять уникальные целые значения, например, значения суррогатного первичного ключа.

Синтаксис для создания столбца типа `serial` таков:

```
CREATE TABLE имя-таблицы ( имя-столбца serial );
```

Эта команда эквивалентна следующей группе команд:

```
CREATE SEQUENCE имя-таблицы_имя-столбца_seq;
CREATE TABLE имя-таблицы
(
  имя-столбца integer NOT NULL
  DEFAULT nextval( 'имя-таблицы_имя-столбца_seq' )
);
ALTER SEQUENCE имя-таблицы_имя-столбца_seq
OWNED BY имя-таблицы.имя-столбца;
```

Для пояснения вышеприведенных команд нам придется немного забежать вперед. Одним из видов объектов в базе данных являются так называемые последовательности. Это, по сути, генераторы уникальных целых чисел. Для работы с этими последовательностями-генераторами используются специальные функции. Одна из них — это функция `nextval`, которая как раз и получает очередное число из последовательности, имя которой указано в качестве параметра функции. В команде `CREATE TABLE` ключевое слово `DEFAULT` предписывает, чтобы СУБД использовала в качестве значения по умолчанию то значение, которое формирует функция `nextval`. Поэтому если в команде вставки строки в таблицу `INSERT INTO` не будет передано значение для поля типа `serial`, то СУБД обратится к услугам этой функции. В том случае, когда в таблице поле типа `serial` является суррогатным первичным ключом, тогда нет необходимости указывать явное значение для вставки в это поле.

В заключение скажем, что кроме типа `serial` существуют еще два аналогичных типа: `bigserial` и `smallserial`. Им фактически, за кадром, соответствуют типы `bigint` и `smallint`. Поэтому при выборе конкретного последовательного типа нужно учитывать предполагаемое число строк в таблице и частоту удаления и вставки строк, поскольку даже для небольшой таблицы может потребоваться большой диапазон, если операции удаления и вставки строк выполняются часто.

4.2. Символьные (строковые) типы

Стандартные представители строковых типов — это типы `character varying(n)` и `character(n)`, где параметр указывает максимальное число символов в строке, которую можно сохранить в столбце такого типа. При работе с многобайтовыми кодировками символов, например UTF-8, нужно учитывать, что речь идет о символах, а не о байтах. Если сохраняемая строка символов будет короче, чем указано в определении типа, то значение типа `character` будет дополнено пробелами до требуемой длины, а значение типа `character varying` будет сохранено так, как есть.

Типы `character varying(n)` и `character(n)` имеют псевдонимы `varchar(n)` и `char(n)` соответственно. На практике, как правило, используют именно эти краткие псевдонимы.

PostgreSQL дополнительно предлагает еще один символьный тип — `text`. В столбец этого типа можно ввести сколь угодно большое значение, конечно, в пределах, установленных при компиляции исходных текстов СУБД.

Документация рекомендует использовать типы `text` и `varchar`, поскольку такое отличительное свойство типа `character`, как дополнение значений пробелами, на практике почти не востребовано. В PostgreSQL обычно используется тип `text`.

Константы символьных типов в SQL-командах заключаются в одинарные кавычки.

```
SELECT 'PostgreSQL';
```

```
?column?  
-----  
PostgreSQL  
(1 строка)
```

В том случае, когда в константе содержится символ одинарной кавычки или обратной косой черты, их необходимо удваивать.

Например:

```
SELECT 'PGDAY' '17';
```

```
?column?  
-----  
PGDAY'17  
(1 строка)
```

В том случае, когда таких символов в константе много, все выражение становится трудно воспринимать. На помощь может прийти использование удвоенного символа «\$». Эти символы выполняют ту же роль, что и одинарные кавычки, когда в них заключается строковая константа. При использовании символов «\$» в качестве ограничителей уже не нужно удваивать никакие символы, содержащиеся в самой константе: ни одинарные кавычки, ни символы обратной косой черты.

Например:

```
SELECT $$PGDAY'17$$;
```

```
?column?  
-----  
PGDAY'17  
(1 строка)
```

Возможность использования символов доллара в роли ограничителей символьной константы не является частью стандарта SQL. Это расширение, предлагаемое PostgreSQL. Подробно об этом написано в разделе документации 4.1.2.4 «Строковые константы, заключенные в доллары».

PostgreSQL предлагает еще одно расширение стандарта SQL — строковые константы в стиле языка C. Чтобы иметь возможность их использовать, нужно перед начальной одинарной кавычкой написать символ «E».

Например, для включения в константу символа новой строки нужно сделать так:

```
SELECT E'PGDAY\n17';
```

```
?column?
-----
PGDAY  +
17
(1 строка)
```

При использовании C-стиля необходимо удваивать обратную косую черту, если требуется поместить ее в константу буквально. А для включения в содержимое константы символа обратной кавычки можно либо удвоить ее, либо сделать так:

```
SELECT E'PGDAY\'17';
```

```
?column?
-----
PGDAY'17
(1 строка)
```

Подробно об использовании C-стиля написано в разделе документации 4.1.2.2 «Строковые константы со спецпоследовательностями в стиле C».

4.3. Типы «дата/время»

PostgreSQL поддерживает все типы данных, предусмотренные стандартом SQL для даты и времени. Даты обрабатываются в соответствии с григорианским календарем, причем это делается даже в тех случаях, когда дата относится к тому моменту времени, когда этот календарь в данной стране еще не был принят. Для этих типов данных предусмотрены определенные форматы для ввода значений и для вывода. Причем эти форматы могут не совпадать. Важно помнить, что при вводе значений их нужно заключать в одинарные кавычки, как и текстовые строки.

Начнем рассмотрение с типа `date`. Рекомендуемый стандартом ISO 8601 формат ввода дат таков: «`yyyy-mm-dd`», где символы «`y`», «`m`» и «`d`» обозначают цифру года, месяца и дня соответственно. PostgreSQL позволяет использовать и другие форматы

для ввода, например: «Sep 12, 2016», что означает 12 сентября 2016 года. При выводе значений PostgreSQL использует формат по умолчанию, если не предписан другой формат. По умолчанию используется формат, рекомендуемый стандартом ISO 8601: «yyyy-mm-dd».

```
SELECT '2016-09-12'::date;
```

```
      date
-----
2016-09-12
(1 строка)
```

А в следующем примере используется другой формат ввода, но формат вывода остается тот же самый, поскольку мы его не изменяли:

```
SELECT 'Sep 12, 2016'::date;
```

```
      date
-----
2016-09-12
(1 строка)
```

Чтобы «сказать» СУБД, что введенное значение является датой, а не простой символьной строкой, мы использовали операцию *приведения типа*. В PostgreSQL она оформляется с использованием двойного символа «двоеточие» и имени того типа, к которому мы приводим данное значение. Важно учесть, что при выполнении приведения типа производится проверка значения на соответствие формату целевого типа и множеству его допустимых значений.

В PostgreSQL предусмотрен целый ряд функций для работы с датами и временем. Например, для получения значения текущей даты служит функция `current_date`. Ее особенностью является то, что при ее вызове круглые скобки не используются.

```
SELECT current_date;
```

```
      date
-----
2016-09-21
(1 строка)
```

Если нам требуется вывести дату в другом формате, то для разового преобразования формата можно использовать функцию `to_char`, например:

```
SELECT to_char( current_date, 'dd-mm-yyyy' );
```

СУБД выведет:

```
to_char
-----
21-09-2016
(1 строка)
```

Обратите внимание, что для демонстрации возможностей СУБД по работе с датами нам даже не потребовалось создавать таблицу, а оказалось достаточно лишь команды `SELECT`.

Для хранения времени суток служат два типа данных: `time` и `time with time zone`. Первый из них хранит только время суток, а второй — дополнительно — еще и часовой пояс. Однако документация на PostgreSQL не рекомендует использовать тип `time with time zone`, поскольку смещение (offset), соответствующее конкретному часовому поясу, может зависеть от даты перехода на летнее время и обратно, но в этом типе дата отсутствует. При вводе значений времени допустимы различные форматы, например:

```
SELECT '21:15'::time;
```

При выводе СУБД дополнит введенное значение, в котором присутствуют только часы и минуты, секундами.

```
time
-----
21:15:00
(1 строка)
```

Чтобы «сказать» СУБД, что введенное значение является значением времени, а не простой символьной строкой, мы опять использовали операцию приведения типа. Предложим СУБД заведомо недопустимое значение времени, например:

```
SELECT '25:15'::time;
```

Получим такое сообщение об ошибке:

```
ОШИБКА: значение поля типа date/time вне диапазона: "25:15"
СТРОКА 1: select '25:15'::time;
          ^
```

А теперь возьмем значение, которое включает еще и секунды.

```
SELECT '21:15:26'::time;
```

```

time
-----
21:15:26
(1 строка)

```

А если число секунд недопустимое, то опять получим сообщение об ошибке.

```
SELECT '21:15:69'::time;
```

```

ОШИБКА: значение поля типа date/time вне диапазона: "21:15:69"
СТРОКА 1: select '21:15:69'::time;
          ^

```

Время можно вводить не только в 24-часовом формате, но и в 12-часовом, при этом нужно использовать дополнительные суффиксы *am* и *pm*. Например:

```
SELECT '10:15:16 am'::time;
```

```

time
-----
10:15:16
(1 строка)

```

```
SELECT '10:15:16 pm'::time;
```

```

time
-----
22:15:16
(1 строка)

```

Для получения значения текущего времени служит функция `current_time`. При ее вызове круглые скобки не используются.

```
SELECT current_time;
```

```

timetz
-----
23:51:57.293522+03
(1 строка)

```

Текущее время выводится с высокой точностью и дополняется числовым значением, соответствующим локальному часовому поясу, который установлен в конфигурационном файле сервера PostgreSQL. В приведенном примере значение часового пояса равно +03, но если ваш компьютер находится в другом часовом поясе, то это значение будет другим, например, для регионов Сибири оно может быть +08.

В результате объединения типов даты и времени получается интегральный тип — временная отметка. Этот тип существует в двух вариантах: с учетом часового пояса — `timestamp with time zone`, либо без учета часового пояса — `timestamp`. Для первого варианта существует сокращенное наименование — `timestampz`, которое является расширением PostgreSQL. При вводе и выводе значений этого типа данных используются соответствующие форматы ввода и вывода даты и времени. Вот пример с учетом часового пояса:

```
SELECT timestamp with time zone '2016-09-21 22:25:35';
```

```
          timestampz
-----
2016-09-21 22:25:35+03
(1 строка)
```

Обратите внимание, что хотя мы не указали явно значение часового пояса при вводе данных, при выводе это значение `+03` было добавлено.

А это пример без учета часового пояса:

```
SELECT timestamp '2016-09-21 22:25:35';
```

```
          timestamp
-----
2016-09-21 22:25:35
(1 строка)
```

В рассмотренных примерах мы использовали синтаксис тип `'строка'` для указания конкретного типа простой литеральной константы. Имя типа мы указывали не после преобразуемого литерала, а перед ним, например, `timestamp '2016-09-21 22:25:35'`. Строго говоря, это не является операцией приведения типа, хотя и похоже на нее. Подробно данный вопрос рассмотрен в разделах документации 4.1.2.7 «Константы других типов» и 4.2.9 «Приведения типов».

Для получения значения текущей временной отметки (т. е. даты и времени в одном значении) служит функция `current_timestamp`. Она также вызывается без указания круглых скобок. Приведем пример ее использования.

```
SELECT current_timestamp;
```

```
          now
-----
2016-09-27 18:27:37.767739+03
(1 строка)
```

Здесь в выводе присутствует и часовой пояс: +03.

Оба типа — `timestamp` и `timestampz` — занимают один и тот же объем 8 байтов, но значения типа `timestampz` хранятся, будучи приведенными к нулевому часовому поясу (UTC), а перед выводом приводятся к часовому поясу пользователя.

На практике при принятии решения о том, какой из этих двух типов — `timestamp` или `timestampz` — использовать, необходимо учитывать, требуется ли значения, хранящиеся в таблице, приводить к местному часовому поясу или не требуется. Например, в расписании авиарейсов указывается местное время как для аэропорта отправления, так и для аэропорта прибытия. Поэтому в таком случае нужно использовать тип `timestamp`, чтобы это время не приводилось к текущему часовому поясу пользователя, где бы он ни находился.

Из двух этих типов данных чаще используется `timestampz`.

Последним типом является `interval`, который представляет продолжительность отрезка времени между двумя моментами времени. Его формат ввода таков:

quantity unit [quantity unit ...] direction

Здесь `unit` означает единицу измерения, а `quantity` — количество таких единиц. В качестве единиц измерения можно использовать следующие: `microsecond`, `millisecond`, `second`, `minute`, `hour`, `day`, `week`, `month`, `year`, `decade`, `century`, `millennium`. Параметр `direction` может принимать значение `ago` (т. е. «тому назад») либо быть пустым. Приведем примеры:

```
SELECT '1 year 2 months ago'::interval;
```

```
interval
-----
-1 years -2 mons
(1 строка)
```

Обратите внимание, что параметр `ago` заставляет СУБД добавить знак «минус» перед всеми полями.

Можно использовать альтернативный формат, предлагаемый стандартом ISO 8601:

P [years-months-days] [T hours:minutes:seconds]

Здесь строка должна начинаться с символа «P», а символ «T» разделяет дату и время (все выражение пишется без пробелов).

Например:

```
SELECT 'P0001-02-03T04:05:06'::interval;
```

```
interval
-----
1 year 2 mons 3 days 04:05:06
(1 строка)
```

Поскольку интервал — это отрезок времени между двумя временными отметками, то значение этого типа можно получить при вычитании одной временной отметки из другой.

```
SELECT ('2016-09-16'::timestamp - '2016-09-01'::timestamp)::interval;
```

```
interval
-----
15 days
(1 строка)
```

Как мы уже говорили ранее, в PostgreSQL предусмотрен целый ряд функций для работы с датами и временем. Например, для получения значений текущей даты, текущего времени и текущей временной отметки (т. е. даты и времени в одном значении) служат функции `current_date`, `current_time`, `current_timestamp`. Эти функции часто применяются для получения значений по умолчанию при вставке строк в таблицы. Их особенностью является то, что при их вызове круглые скобки не используются. Для получения полной информации обратитесь к документации (раздел 9.9 «Операторы и функции даты/времени»).

Значения временных отметок можно усекавать с той или иной точностью с помощью функции `date_trunc`. Например, для получения текущей временной отметки с точностью до одного часа нужно сделать так:

```
SELECT ( date_trunc( 'hour', current_timestamp ) );
```

```
date_trunc
-----
2016-09-27 22:00:00+03
(1 строка)
```

Из значений временных отметок можно с помощью функции `extract` извлекать отдельные поля, т. е. год, месяц, день, число часов, минут или секунд и т. д. Например, чтобы извлечь номер месяца, нужно сделать так:

```
SELECT extract( 'mon' FROM timestamp '1999-11-27 12:34:56.123459' );
```

```
date_part
-----
11
(1 строка)
```

Напомним, что выражение `timestamp '1999-11-27 12:34:56.123459'` не означает операцию приведения типа. Оно присваивает тип данных `timestamp` литеральной константе.

4.4. Логический тип

Логический (`boolean`) тип может принимать три состояния: истина и ложь, а также неопределенное состояние, которое можно представить значением `NULL`. Таким образом, тип `boolean` реализует трехзначную логику.

В качестве истинного состояния могут служить следующие значения: `TRUE`, `'t'`, `'true'`, `'y'`, `'yes'`, `'on'`, `'1'`.

В качестве ложного состояния могут служить следующие значения: `FALSE`, `'f'`, `'false'`, `'n'`, `'no'`, `'off'`, `'0'`.

Для примера создадим таблицу с двумя столбцами и добавим в нее несколько строк следующим образом:

```
CREATE TABLE databases ( is_open_source boolean, dbms_name text );

INSERT INTO databases VALUES ( TRUE, 'PostgreSQL' );
INSERT INTO databases VALUES ( FALSE, 'Oracle' );
INSERT INTO databases VALUES ( TRUE, 'MySQL' );
INSERT INTO databases VALUES ( FALSE, 'MS SQL Server' );
```

Теперь выполним выборку всех строк из этой таблицы:

```
SELECT * FROM databases;
```

```
is_open_source | dbms_name
-----+-----
t              | PostgreSQL
f              | Oracle
t              | MySQL
f              | MS SQL Server
(4 строки)
```

Выберем только СУБД с открытым исходным кодом:

```
SELECT * FROM databases WHERE is_open_source;
```

is_open_source	dbms_name
t	PostgreSQL
t	MySQL

(2 строки)

Обратите внимание, что в условии WHERE для проверки логических значений можно не писать выражение `WHERE is_open_source = 'yes'`, а достаточно просто указать имя столбца, содержащего логическое значение: `WHERE is_open_source`.

4.5. Массивы

PostgreSQL позволяет создавать в таблицах такие столбцы, в которых будут содержаться не скалярные значения, а массивы переменной длины. Эти массивы могут быть многомерными и могут содержать значения любого из встроенных типов, а также типов данных, определенных пользователем.

Предположим, что нам необходимо сформировать и сохранить в базе данных в удобной форме графики работы пилотов авиакомпании, т. е. номера дней недели, когда они совершают полеты. Создадим таблицу, в которой эти графики будут храниться в виде единых списков, т. е. в виде одномерных массивов.

```
CREATE TABLE pilots  
(  
    pilot_name text,  
    schedule integer[]  
);
```

```
CREATE TABLE
```

Для указания на то, что это массив, нужно добавить квадратные скобки к наименованию типа данных. При этом задавать число элементов не обязательно.

Давайте добавим в таблицу четыре строки. Массив в команде вставки представлен в виде строкового литерала с указанием типа данных и квадратных скобок, обозначающих массив. Обратите внимание, что все массивы имеют различное число элементов.

```
INSERT INTO pilots
VALUES ( 'Ivan',  '{ 1, 3, 5, 6, 7 }'::integer[] ),
      ( 'Petr',   '{ 1, 2, 5, 7   }'::integer[] ),
      ( 'Pavel',  '{ 2, 5          }'::integer[] ),
      ( 'Boris',  '{ 3, 5, 6        }'::integer[] );
```

```
INSERT 0 4
```

Посмотрим, что получилось:

```
SELECT * FROM pilots;
```

pilot_name	schedule
Ivan	{1,3,5,6,7}
Petr	{1,2,5,7}
Pavel	{2,5}
Boris	{3,5,6}

(4 строки)

Предположим, что руководство компании решило, что каждый пилот должен летать 4 раза в неделю. Значит, нам придется обновить значения в таблице. Пилоту по имени Boris добавим один день с помощью операции конкатенации:

```
UPDATE pilots
SET schedule = schedule || 7
WHERE pilot_name = 'Boris';
```

```
UPDATE 1
```

Пилоту по имени Pavel добавим один день в конец списка (массива) с помощью функции `array_append`:

```
UPDATE pilots
SET schedule = array_append( schedule, 6 )
WHERE pilot_name = 'Pavel';
```

```
UPDATE 1
```

Ему же добавим один день в начало списка с помощью функции `array_prepend` (обратите внимание, что параметры функции поменялись местами):

```
UPDATE pilots
SET schedule = array_prepend( 1, schedule )
WHERE pilot_name = 'Pavel';
```

```
UPDATE 1
```

У пилота по имени Ivan имеется лишний день в графике. С помощью функции `array_remove` удалим из графика пятницу (второй параметр функции указывает значение элемента массива, а не индекс):

```
UPDATE pilots
  SET schedule = array_remove( schedule, 5 )
  WHERE pilot_name = 'Ivan';
```

UPDATE 1

У пилота по имени Petr изменим дни полетов, не изменяя их общего количества. Воспользуемся индексами для работы на уровне отдельных элементов массива. По умолчанию нумерация индексов начинается с единицы, а не с нуля. При необходимости ее можно изменить. К элементам одного и того же массива можно обращаться в предложении SET по отдельности, как будто это разные столбцы.

```
UPDATE pilots
  SET schedule[ 1 ] = 2, schedule[ 2 ] = 3
  WHERE pilot_name = 'Petr';
```

UPDATE 1

А можно было бы, используя срез (slice) массива, сделать и так:

```
UPDATE pilots
  SET schedule[ 1:2 ] = ARRAY[ 2, 3 ]
  WHERE pilot_name = 'Petr';
```

UPDATE 1

В вышеприведенной команде запись `1:2` означает индексы первого и последнего элементов диапазона массива. Нотация с использованием ключевого слова `ARRAY` — это альтернативный способ создания массива (он соответствует стандарту SQL). Таким образом, присваивание новых значений производится сразу целому диапазону элементов массива.

```
SELECT * FROM pilots;
```

pilot_name	schedule
Boris	{3,5,6,7}
Pavel	{1,2,5,6}
Ivan	{1,3,6,7}
Petr	{2,3,5,7}

(4 строки)

Теперь продемонстрируем основные операции, которые можно применять к массивам, выполняя выборки из таблиц. Получим список пилотов, летающих по средам:

```
SELECT * FROM pilots
WHERE array_position( schedule, 3 ) IS NOT NULL;
```

pilot_name	schedule
Boris	{3,5,6,7}
Ivan	{1,3,6,7}
Petr	{2,3,5,7}

(3 строки)

Функция `array_position` возвращает индекс первого вхождения элемента с указанным значением в массив. Если же такого элемента нет, она возвратит `NULL`. Выберем пилотов, летающих по понедельникам и воскресеньям:

```
SELECT * FROM pilots
WHERE schedule @> '{ 1, 7 }'::integer[];
```

pilot_name	schedule
Ivan	{1,3,6,7}

(1 строка)

Оператор `@>` означает проверку того факта, что в левом массиве содержатся все элементы правого массива. Конечно, при этом в левом массиве могут находиться и другие элементы, что мы и видим в графике этого пилота.

Еще аналогичный вопрос: кто летает по вторникам и/или по пятницам? Для получения ответа воспользуемся оператором `&&`, который проверяет наличие общих элементов у массивов, т. е. пересекаются ли их множества значений. В нашем примере число общих элементов, если они есть, может быть равно одному или двум. Здесь мы использовали нотацию с ключевым словом `ARRAY`, а не `'{2, 5}'::integer[]`. Вы можете применять ту, которая принята в рамках выполнения вашего проекта.

```
SELECT * FROM pilots
WHERE schedule && ARRAY[ 2, 5 ];
```

pilot_name	schedule
Boris	{3,5,6,7}
Pavel	{1,2,5,6}
Petr	{2,3,5,7}

(3 строки)

Сформулируем вопрос в форме отрицания: кто не летает ни во вторник, ни в пятницу? Для получения ответа добавим в предыдущую SQL-команду отрицание NOT:

```
SELECT * FROM pilots
  WHERE NOT ( schedule && ARRAY[ 2, 5 ] );
```

```
pilot_name | schedule
-----+-----
Ivan       | {1,3,6,7}
(1 строка)
```

Иногда требуется развернуть массив в виде столбца таблицы. В таком случае поможет функция unnest:

```
SELECT unnest( schedule ) AS days_of_week
  FROM pilots
  WHERE pilot_name = 'Ivan';
```

```
days_of_week
-----
1
3
6
7
(4 строки)
```

Подробно использование массивов рассмотрено в документации в разделах 8.15 «Массивы» и 9.18 «Функции и операторы для работы с массивами».

4.6. Типы JSON

Типы JSON предназначены для сохранения в столбцах таблиц базы данных таких значений, которые представлены в формате JSON (JavaScript Object Notation). Существует два типа: json и jsonb. Основное различие между ними заключается в быстродействии. Если столбец имеет тип json, тогда сохранение значений происходит быстрее, потому что они записываются в том виде, в котором были введены. Но при последующем использовании этих значений в качестве операндов или параметров функций будет каждый раз выполняться их разбор, что замедляет работу. При использовании типа jsonb разбор производится однократно, при записи значения в таблицу. Это несколько замедляет операции вставки строк, в которых содержатся значения данного типа. Но все последующие обращения к сохраненным значениям выполняются быстрее, т. к. выполнять их разбор уже не требуется.

Есть еще ряд отличий, в частности, тип `json` сохраняет порядок следования ключей в объектах и повторяющиеся значения ключей, а тип `jsonb` этого не делает. Рекомендуется в приложениях использовать тип `jsonb`, если только нет каких-то особых аргументов в пользу выбора типа `json`.

Для иллюстрации использования типов JSON обратимся к тематике авиаперевозок. Предположим, что руководство авиакомпании всемерно поддерживает стремление пилотов улучшать свое здоровье, повышать уровень культуры и расширять кругозор. Поэтому разработчики базы данных авиакомпании получили задание создать специальную таблицу, в которую будут заноситься сведения о тех видах спорта, которыми занимается пилот, будет отмечаться наличие у него домашней библиотеки, а также фиксироваться количество стран, которые он посетил в ходе туристических поездок.

```
CREATE TABLE pilot_hobbies
(
    pilot_name text,
    hobbies jsonb
);

CREATE TABLE

INSERT INTO pilot_hobbies
VALUES ( 'Ivan',
        '{ "sports": [ "футбол", "плавание" ],
          "home_lib": true, "trips": 3
        }'::jsonb
    ),
    ( 'Petr',
        '{ "sports": [ "теннис", "плавание" ],
          "home_lib": true, "trips": 2
        }'::jsonb
    ),
    ( 'Pavel',
        '{ "sports": [ "плавание" ],
          "home_lib": false, "trips": 4
        }'::jsonb
    ),
    ( 'Boris',
        '{ "sports": [ "футбол", "плавание", "теннис" ],
          "home_lib": true, "trips": 0
        }'::jsonb
    );

INSERT 0 4
```

```
SELECT * FROM pilot_hobbies;
```

pilot_name	hobbies
Ivan	{ "trips": 3, "sports": ["футбол", "плавание"], "home_lib": true }
Petr	{ "trips": 2, "sports": ["теннис", "плавание"], "home_lib": true }
Pavel	{ "trips": 4, "sports": ["плавание"], "home_lib": false }
Boris	{ "trips": 0, "sports": ["футбол", "плавание", "теннис"], "home_lib": true }

(4 строки)

Как видно, при выводе строк из таблицы порядок ключей в JSON-объектах не был сохранен.

Предположим, что нужно сформировать футбольную сборную команду нашей авиакомпании для участия в турнире. Мы можем выбрать всех футболистов таким способом:

```
SELECT * FROM pilot_hobbies
```

```
WHERE hobbies @> '{ "sports": [ "футбол" ] }'::jsonb;
```

pilot_name	hobbies
Ivan	{ "trips": 3, "sports": ["футбол", "плавание"], "home_lib": true }
Boris	{ "trips": 0, "sports": ["футбол", "плавание", "теннис"], "home_lib": true }

(2 строки)

Можно было эту задачу решить и таким способом:

```
SELECT pilot_name, hobbies->'sports' AS sports
```

```
FROM pilot_hobbies
```

```
WHERE hobbies->'sports' @> '[ "футбол" ]'::jsonb;
```

pilot_name	sports
Ivan	["футбол", "плавание"]
Boris	["футбол", "плавание", "теннис"]

(2 строки)

В этом решении мы выводим только информацию о спортивных предпочтениях пилотов. Внимательно посмотрите, как используются одинарные и двойные кавычки. Операция `->` служит для обращения к конкретному ключу JSON-объекта.

При создании столбца с типом данных `json` или `jsonb` не требуется задавать структуру объектов, т. е. конкретные имена ключей. Поэтому в принципе возможна ситуация, когда в разных строках в JSON-объектах будут использоваться различные наборы ключей. В нашем примере структуры JSON-объектов во всех строках совпадают. А если бы они не совпадали, то как можно было бы проверить наличие ключа? Продемонстрируем это.

Ключа `sport` в наших объектах нет. Что покажет вызов функции `count`?

```
SELECT count( * )
  FROM pilot_hobbies
 WHERE hobbies ? 'sport';
```

```
count
-----
      0
(1 строка)
```

А вот ключ `sports` присутствует. Выполним ту же проверку:

```
SELECT count( * )
  FROM pilot_hobbies
 WHERE hobbies ? 'sports';
```

Да, так и есть. Такие записи найдены.

```
count
-----
      4
(1 строка)
```

А как выполнять обновление JSON-объектов в строках таблицы? Предположим, что пилот по имени Boris решил посвятить себя только хоккею. Тогда в базе данных мы выполним такую операцию:

```
UPDATE pilot_hobbies
  SET hobbies = hobbies || '{ "sports": [ "хоккей" ] }'
 WHERE pilot_name = 'Boris';
```

```
UPDATE 1
```

Проверим, что получилось:

```
SELECT pilot_name, hobbies
FROM pilot_hobbies
WHERE pilot_name = 'Boris';
```

pilot_name	hobbies
Boris	{ "trips": 0, "sports": ["хоккей"], "home_lib": true }

(1 строка)

Если впоследствии Boris захочет возобновить занятия футболом, то с помощью функции `jsonb_set` можно будет обновить сведения о нем в таблице:

```
UPDATE pilot_hobbies
SET hobbies = jsonb_set( hobbies, '{ sports, 1 }', '"футбол"' )
WHERE pilot_name = 'Boris';
```

UPDATE 1

Второй параметр функции указывает путь в пределах JSON-объекта, куда нужно добавить новое значение. В данном случае этот путь состоит из имени ключа (`sports`) и номера добавляемого элемента в массиве видов спорта (номер 1). Нумерация элементов начинается с нуля. Третий параметр имеет тип `jsonb`, поэтому его литерал заключается в одинарные кавычки, а само добавляемое значение берется в двойные кавычки. В результате получается — `'"футбол"'`.

Проверим успешность выполнения этой операции:

```
SELECT pilot_name, hobbies
FROM pilot_hobbies
WHERE pilot_name = 'Boris';
```

pilot_name	hobbies
Boris	{ "trips": 0, "sports": ["хоккей", "футбол"], "home_lib": true }

(1 строка)

Подробно использование типов JSON рассмотрено в документации в разделах 8.14 «Типы JSON» и 9.15 «Функции и операторы JSON».

Контрольные вопросы и задания

1. Создайте таблицу, содержащую атрибут типа `numeric(precision, scale)`. Пусть это будет таблица, содержащая результаты каких-то измерений.

Команда может быть, например, такой:

```
CREATE TABLE test_numeric  
( measurement numeric(5, 2),  
  description text  
);
```

Попробуйте с помощью команды `INSERT` продемонстрировать округление вводимого числа до той точности, которая задана при создании таблицы.

Подумайте, какая из следующих команд вызовет ошибку и почему? Проверьте свои предположения, выполнив эти команды.

```
INSERT INTO test_numeric  
  VALUES ( 999.9999, 'Какое-то измерение ' );  
INSERT INTO test_numeric  
  VALUES ( 999.9009, 'Еще одно измерение' );  
INSERT INTO test_numeric  
  VALUES ( 999.1111, 'И еще измерение' );  
INSERT INTO test_numeric  
  VALUES ( 998.9999, 'И еще одно' );
```

Продемонстрируйте генерирование ошибки при попытке ввода числа, количество цифр в котором слева от десятичной точки (запятой) превышает допустимое.

2. Предположим, что возникла необходимость хранить в одном столбце таблицы данные, представленные с различной точностью. Это могут быть, например, результаты физических измерений разнородных показателей или различные медицинские показатели здоровья пациентов (результаты анализов). В таком случае можно использовать тип `numeric` без указания масштаба и точности.

Команда для создания таблицы может быть, например, такой:

```
CREATE TABLE test_numeric  
( measurement numeric,  
  description text  
);
```

Если у вас в базе данных уже есть таблица с таким же именем, то можно предварительно ее удалить с помощью команды

```
DROP TABLE test_numeric;
```

Вставьте в таблицу несколько строк:

```
INSERT INTO test_numeric  
VALUES ( 1234567890.0987654321,  
        'Точность 20 знаков, масштаб 10 знаков' );
```

```
INSERT INTO test_numeric  
VALUES ( 1.5,  
        'Точность 2 знака, масштаб 1 знак' );
```

```
INSERT INTO test_numeric  
VALUES ( 0.12345678901234567890,  
        'Точность 21 знак, масштаб 20 знаков' );
```

```
INSERT INTO test_numeric  
VALUES ( 1234567890,  
        'Точность 10 знаков, масштаб 0 знаков (целое число)' );
```

Теперь сделайте выборку из таблицы и посмотрите, что все эти разнообразные значения сохранены именно в том виде, как вы их вводили.

3. Тип данных `numeric` поддерживает специальное значение `NaN`, которое означает «не число» (`not a number`). В документации утверждается, что значение `NaN` считается равным другому значению `NaN`, а также что значение `NaN` считается большим любого другого «нормального» значения, т. е. не-`NaN`. Проверьте эти утверждения с помощью SQL-команды `SELECT`.

В качестве примера приведем команду:

```
SELECT 'NaN'::numeric > 10000;
```

```
?column?  
-----  
t  
(1 строка)
```

4. При работе с числами типов `real` и `double precision` нужно помнить, что сравнение двух чисел с плавающей точкой на предмет равенства их значений может привести к неожиданным результатам.

Например, сравним два очень маленьких числа (они представлены в экспоненциальной форме записи):

```
SELECT '5e-324'::double precision > '4e-324'::double precision;
```

```
?column?
-----
f
(1 строка)
```

Чтобы понять, почему так получается, выполните еще два запроса.

```
SELECT '5e-324'::double precision;
```

```
float8
-----
4.94065645841247e-324
(1 строка)
```

```
SELECT '4e-324'::double precision;
```

```
float8
-----
4.94065645841247e-324
(1 строка)
```

Самостоятельно проведите аналогичные эксперименты с очень большими числами, находящимися на границе допустимого диапазона для чисел типов `real` и `double precision`.

- Типы данных `real` и `double precision` поддерживают специальные значения `Infinity` (бесконечность) и `-Infinity` (отрицательная бесконечность). Проверьте с помощью SQL-команды `SELECT` ожидаемые свойства этих значений. Например, сравните `Infinity` с наибольшим значением, которое допускается для типа `double precision` (можно использовать сокращенное написание `Inf`):

```
SELECT 'Inf'::double precision > 1E+308;
```

```
?column?
-----
t
(1 строка)
```

Выполните аналогичный запрос для наименьшего возможного значения типа `double precision`.

6. Типы данных `real` и `double precision` поддерживают специальное значение `NaN`, которое означает «не число» (not a number).

В математике существует такое понятие, как *неопределенность*. В качестве одного из ее вариантов служит результат операции умножения нуля на бесконечность. Посмотрите, что выдаст в результате PostgreSQL:

```
SELECT 0.0 * 'Inf'::real;
```

```
?column?
-----
NaN
(1 строка)
```

В документации утверждается, что значение `NaN` считается равным другому значению `NaN`, а также что значение `NaN` считается бóльшим любого другого «нормального» значения, т. е. не-`NaN`. Проверьте эти утверждения с помощью SQL-команды `SELECT`.

Например, сравните значения `NaN` и `Infinity`.

```
select 'NaN'::real > 'Inf'::real;
```

```
?column?
-----
t
(1 строка)
```

7. Тип `serial` может применяться для столбцов, содержащих числовые значения, которые должны быть уникальными в пределах таблицы, например, идентификаторы каких-то объектов. В качестве иллюстрации применения типа `serial` предложим таблицу, содержащую наименования улиц и площадей:

```
CREATE TABLE test_serial
( id serial,
  name text
);
```

Введите несколько строк. Обратите внимание, что значение для столбца `id` указывать не обязательно (и даже не нужно). Но поскольку мы задаем значения не для всех столбцов, имеющих в таблице, мы должны указать в команде `INSERT` не только список значений, но и список столбцов. Конечно, в данном простом случае эти списки состоят лишь из одного элемента.

```
INSERT INTO test_serial ( name ) VALUES ( 'Вишневая' );  
INSERT INTO test_serial ( name ) VALUES ( 'Грушевая' );  
INSERT INTO test_serial ( name ) VALUES ( 'Зеленая' );
```

Сделайте выборку данных из таблицы, вы увидите, что значения столбца `id` имеют последовательные значения, начиная с 1.

Давайте проведем эксперимент со столбцом `id`. Выполните команду `INSERT`, в которой укажите явное значение столбца `id`:

```
INSERT INTO test_serial ( id, name ) VALUES ( 10, 'Прохладная' );
```

А теперь добавьте еще одну строку, но уже не указывая явно значение для столбца `id` (как мы поступали в предыдущих командах):

```
INSERT INTO test_serial ( name ) VALUES ( 'Луговая' );
```

Вы увидите, что явное задание значения для столбца `id` не влияет на автоматическое генерирование значений этого столбца.

8. Немного усложним определение таблицы из предыдущего задания. Пусть теперь столбец `id` будет первичным ключом этой таблицы.

```
CREATE TABLE test_serial  
( id serial PRIMARY KEY,  
  name text  
);
```

Теперь выполните следующие команды для добавления строк в таблицу и удаления одной строки из нее. Для пошагового управления этим процессом выполняйте выборку данных из таблицы с помощью команды `SELECT` после каждой команды вставки или удаления.

```
INSERT INTO test_serial ( name ) VALUES ( 'Вишневая' );
```

Явно зададим значение столбца `id`:

```
INSERT INTO test_serial ( id, name ) VALUES ( 2, 'Прохладная' );
```

При выполнении этой команды СУБД выдаст сообщение об ошибке. Почему?

```
INSERT INTO test_serial ( name ) VALUES ( 'Грушевая' );
```

Повторим эту же команду. Теперь все в порядке. Почему?

```
INSERT INTO test_serial ( name ) VALUES ( 'Грушевая' );
```

Добавим еще одну строку.

```
INSERT INTO test_serial ( name ) VALUES ( 'Зеленая' );
```

А теперь удалим ее же.

```
DELETE FROM test_serial WHERE id = 4;
```

Добавим последнюю строку.

```
INSERT INTO test_serial ( name ) VALUES ( 'Луговая' );
```

Теперь сделаем выборку.

```
SELECT * FROM test_serial;
```

Вы увидите, что в нумерации образовалась «дыра». Это из-за того, что при формировании нового значения из последовательности поиск максимального значения, уже имеющегося в столбце, не выполняется.

```
id | name
---+-----
 1 | Вишневая
 2 | Прохладная
 3 | Грушевая
 5 | Луговая
(4 строки)
```

9. Какой календарь используется в PostgreSQL для работы с датами: юлианский или григорианский?
10. Каждый тип данных из группы «дата/время» имеет ограничение на минимальное и максимальное допустимое значение. Найдите в документации в разделе 8.5 «Типы даты/времени» эти значения и подумайте, почему они таковы.
11. Типы `timestamp`, `time` и `interval` позволяют задать точность ввода и вывода значений. Точность предписывает количество десятичных цифр в поле секунд. Проиллюстрируем эту возможность на примере типа `time`, выполнив три запроса: в первом запросе вообще не используем параметр точности, во втором назначим его равным 0, в третьем запросе сделаем его равным 3.

```
SELECT current_time;
```

```
        timetz
-----
19:46:14.584641+03
(1 строка)
```

```
SELECT current_time::time( 0 );
```

```
        time
-----
19:39:45
(1 строка)
```

```
SELECT current_time::time( 3 );
```

```
        time
-----
19:39:54.085
(1 строка)
```

Выполните подобные команды для типов `timestamp` и `interval`.

Тип `date` такой возможности — задавать точность — не имеет. Как вы думаете, почему?

- 12.* Формат ввода и вывода даты можно изменить с помощью конфигурационного параметра `datestyle`. Значение этого параметра состоит из двух компонентов: первый управляет форматом вывода даты, а второй регулирует порядок следования составных частей даты (год, месяц, день) при вводе и выводе. Текущее значение этого параметра можно узнать с помощью команды `SHOW`:

```
SHOW datestyle;
```

По умолчанию он имеет такое значение:

```
        DateStyle
-----
ISO, DMY
(1 строка)
```

Продemonстрируем влияние этого параметра на работу с типами данных `date` и `timestamp`. Для экспериментов возьмем дату, в которой число (день) превышает 12, чтобы нельзя было день перепутать с номером месяца. Пусть это будет, например, 18 мая 2016 г.

```
SELECT '18-05-2016'::date;
```

Хотя порядок следования составных частей даты задан в виде DMY, т. е. «день, месяц, год», но при выводе он изменяется на «год, месяц, день».

```
      date
-----
 2016-05-18
(1 строка)
```

Попробуем ввести дату в порядке «месяц, день, год»:

```
SELECT '05-18-2016'::date;
```

В ответ получим сообщение об ошибке. Если бы мы выбрали дату, в которой число (день) было бы не больше 12, например, 9, то сообщение об ошибке не было бы сформировано, т. е. мы с такой датой не смогли бы проиллюстрировать влияние значения DMY параметра `datestyle`. Но главное, что в таком случае мы бы просто не заметили допущенной ошибки.

А вот использовать порядок «год, месяц, день» при вводе можно несмотря на то, что параметр `datestyle` предписывает «день, месяц, год». Порядок «год, месяц, день» является универсальным, его можно использовать всегда, независимо от настроек параметра `datestyle`.

```
SELECT '2016-05-18'::date;
```

```
      date
-----
 2016-05-18
(1 строка)
```

Продолжим экспериментирование с параметром `datestyle`. Давайте изменим его значение. Сделать это можно многими способами, но мы упомянем лишь некоторые:

- изменив его значение в конфигурационном файле `postgresql.conf`, который в нашей инсталляции PostgreSQL, описанной в главе 2, находится в каталоге `/usr/local/pgsql/data`;
- назначив переменную системного окружения `PGDATESTYLE`;
- воспользовавшись командой `SET`.

Сейчас выберем третий способ, а первые два рассмотрим при выполнении других заданий. Поскольку параметр `datestyle` состоит фактически из двух частей, которые можно задавать не только обе сразу, но и по отдельности, изменим только порядок следования составных частей даты, не изменяя формат вывода с ISO на какой-либо другой.

```
SET datestyle TO 'MDY';
```

Повторим одну из команд, выполненных ранее. Теперь она должна вызвать ошибку. Почему?

```
SELECT '18-05-2016'::date;
```

А такая команда, наоборот, теперь будет успешно выполнена:

```
SELECT '05-18-2016'::date;
```

Теперь приведите настройку параметра `datestyle` в исходное состояние:

```
SET datestyle TO DEFAULT;
```

Самостоятельно выполните команды `SELECT`, приведенные выше, но замените в них тип `date` на тип `timestamp`. Вы увидите, что дата в рамках типа `timestamp` обрабатывается аналогично типу `date`.

Сейчас изменим сразу обе части параметра `datestyle`:

```
SET datestyle TO 'Postgres, DMY';
```

Проверьте полученный результат с помощью команды `SHOW`.

Самостоятельно выполните команды `SELECT`, приведенные выше, как для значения типа `date`, так и для значения типа `timestamp`. Обратите внимание, что если выбран формат `Postgres`, то порядок следования составных частей даты (день, месяц, год), заданный в параметре `datestyle`, используется не только при вводе значений, но и при выводе. Напомним, что вводом мы считаем команду `SELECT`, а выводом — результат ее выполнения, выведенный на экран.

В документации (см. раздел 8.5.2 «Вывод даты/времени») сказано, что формат вывода даты может принимать значения `ISO`, `Postgres`, `SQL` и `German`. Первые два варианта мы уже рассмотрели. Самостоятельно поэкспериментируйте с двумя оставшимися по той же схеме, по которой вы уже действовали ранее при выполнении этого задания. Можно воспользоваться и стандартными функциями `current_date` и `current_timestamp`.

13. Установить новое значение параметра `datestyle` можно с помощью создания переменной системного окружения `PGDATESTYLE`. Назначить эту переменную можно в конфигурационных файлах операционной системы. Но если нам нужно сделать это только на время текущего сеанса работы клиентской программы, например утилиты `psql`, то можно ввести значение этой переменной непосредственно в командной строке:

```
PGDATESTYLE="Postgres" psql -d test -U имя-пользователя
```

Прodelайте эти действия, а затем уже из командной строки утилиты `psql` проверьте текущее значение параметра `datestyle` с помощью команды `SHOW`.

14. Назначить значение параметра `datestyle` можно в конфигурационном файле `postgresql.conf`, который находится в каталоге `/usr/local/pgsql/data`. Предварительно сохраните текущую (корректно работающую) версию этого файла, а затем измените в нем значение параметра `datestyle`, например, на `Postgres, YMD`. Перезапустите сервер PostgreSQL, чтобы изменения вступили в силу.

Для проверки полученного результата выполните несколько команд `SELECT`, например:

```
SELECT '05-18-2016'::timestamp;  
SELECT current_timestamp;
```

15. В документации в разделе 9.8 «Функции форматирования данных» представлены описания множества полезных функций, позволяющих преобразовать в строку данные других типов, например, `timestamp`. Одна из таких функций — `to_char`.

Приведем несколько команд, иллюстрирующих использование этой функции. Ее первым параметром является формируемое значение, а вторым — шаблон, описывающий формат, в котором это значение будет представлено при вводе или выводе. Сначала попробуйте разобраться, не обращаясь к документации, в том, что означает второй параметр этой функции в каждой из приведенных команд, а затем проверьте свои предположения по документации.

```
SELECT to_char( current_timestamp, 'mi:ss' );
```

```
to_char  
-----  
47:43  
(1 строка)
```

```
SELECT to_char( current_timestamp, 'dd' );
```

```
to_char
-----
12
(1 строка)
```

```
SELECT to_char( current_timestamp, 'yyyy-mm-dd' );
```

```
to_char
-----
2017-03-12
(1 строка)
```

Поэкспериментируйте с этой функцией, извлекая из значения типа timestamp различные поля и располагая их в нужном вам порядке.

16. При выполнении приведения типа данных производится проверка значения на допустимость. Попробуйте ввести недопустимое значение даты, например, 29 февраля в невисокосном году.

```
SELECT 'Feb 29, 2015'::date;
```

Получите сообщение об ошибке.

17. При выполнении приведения типа данных производится проверка значения на допустимость. Попробуйте ввести недопустимое значение времени, например, с нарушением формата.

```
SELECT '21:15:16:22'::time;
```

```
ОШИБКА: неверный синтаксис для типа time: "21:15:16:22"
СТРОКА 1: select '21:15:16:22'::time;
          ^
```

18. Как вы думаете, значение какого типа будет получено при вычитании одной даты из другой?

Например:

```
SELECT ( '2016-09-16'::date - '2016-09-01'::date );
```

Сначала попробуйте получить ответ, рассуждая логически, а затем проверьте на практике в утилите psql.

19. С типами даты и времени можно выполнять различные арифметические операции. Как правило, их применение является интуитивно понятным. Выполните следующую команду и проанализируйте результат.

```
SELECT ( '20:34:35'::time - '19:44:45'::time );
```

А теперь попробуйте предположить, какой результат будет получен, если в этой команде знак «минус» заменить на знак «плюс»? Проверьте ваши предположения с помощью утилиты `psql`. Подробное описание всех допустимых арифметических операций с датами и временем приведено в документации в разделе 9.9 «Операторы и функции даты/времени».

20. Значение типа `interval` можно получить при вычитании одной временной отметки из другой, например:

```
SELECT ( current_timestamp - '2016-01-01'::timestamp )  
AS new_date;
```

```
new_date  
-----  
278 days 00:10:33.33236  
(1 строка)
```

А что получится, если прибавить интервал к временной отметке? Сначала попробуйте дать ответ, не прибегая к помощи утилиты `psql`, а затем проверьте свой ответ с помощью этой утилиты. Например, прибавим интервал длительностью в 1 месяц к текущей к временной отметке:

```
SELECT ( current_timestamp + '1 mon'::interval ) AS new_date;
```

В этой команде с помощью ключевого слова `AS` мы назначили псевдоним для того столбца, который будет выведен в результате. Выполните эту же команду, убрав псевдоним, и найдите отличия.

21. Можно с высокой степенью уверенности предположить, что при прибавлении интервалов к датам и временным отметкам PostgreSQL учитывает тот факт, что различные месяцы имеют различное число дней. Но как это реализуется на практике? Например, что получится при прибавлении интервала в 1 месяц к последнему дню января и к последнему дню февраля? Сначала сделайте обоснованные предположения о результатах следующих двух команд, а затем проверьте предположения на практике и проанализируйте полученные результаты:

```
SELECT ( '2016-01-31'::date + '1 mon'::interval ) AS new_date;  
SELECT ( '2016-02-29'::date + '1 mon'::interval ) AS new_date;
```

22. Форматом ввода и вывода интервалов управляет параметр `interval style`. Его можно изменить с помощью способов, аналогичных тем, что были описаны выше для параметра `date style`. Самостоятельно поэкспериментируйте с различными значениями параметра `interval style` аналогично тому, как вы это делали с параметром `date style`. Используйте раздел 8.5 «Типы даты/времени» в документации.

Напомним, что вернуть исходное значение этого параметра в `psql` можно с помощью команды

```
SET intervalstyle TO DEFAULT;
```

23. Выполните следующие две команды и объясните различия в выведенных результатах:

```
SELECT ( '2016-09-16'::date - '2015-09-01'::date );  
SELECT ( '2016-09-16'::timestamp - '2015-09-01'::timestamp );
```

24. Выполните следующие две команды и объясните различия в выведенных результатах:

```
SELECT ( '20:34:35'::time - 1 );  
SELECT ( '2016-09-16'::date - 1 );
```

Почему при выполнении первой команды возникает ошибка? Как можно модифицировать эту команду, чтобы ошибка исчезла?

Для получения полной информации обратитесь к разделу 9.9 «Операторы и функции даты/времени» документации.

25. Значения временных отметок можно усекать с той или иной точностью с помощью функции `date_trunc`. Например, с помощью следующей команды можно «отрезать» дробную часть секунды:

```
SELECT ( date_trunc( 'sec',  
                timestamp '1999-11-27 12:34:56.987654' ) );
```

```
    date_trunc  
-----  
1999-11-27 12:34:56  
(1 строка)
```

Напомним, что в данной команде используется операция приведения типа.

Выполните эту команду, последовательно указывая в качестве первого параметра значения `microsecond`, `millisecond`, `second`, `minute`, `hour`, `day`, `week`, `month`, `year`, `decade`, `century`, `millennium` (которые обозначают соответственно микросекунды, миллисекунды, секунды, минуты, часы, дни, недели, месяцы, годы, десятилетия, века и тысячелетия). Допустимы сокращения `sec`, `min`, `mon`, `dec`, `cent`, `mil`.

Обратите внимание, что результирующее значение получается не путем округления исходного значения, а именно путем отбрасывания более мелких единиц. При этом поля времени (часы, минуты и секунды) заменяются нулями, а поля даты (годы, месяцы и дни) — заменяются цифрами 01. Однако при использовании параметра `week` картина получается более интересная.

26. Функция `date_trunc` может работать не только с данными типа `timestamp`, но также и с данными типа `interval`. Самостоятельно ознакомьтесь с этими возможностями по документации (см. раздел 9.9 «Операторы и функции даты/времени»).
27. Весьма полезной является функция `extract`. С ее помощью можно извлечь значение отдельного поля из временной отметки `timestamp`. Наименование поля задается в первом параметре. Эти наименования такие же, что и для функции `date_trunc`. Выполните следующую команду

```
SELECT extract(  
    'microsecond' from timestamp '1999-11-27 12:34:56.123459'  
);
```

Она выводит не просто значение поля микросекунд, т. е. 123459, а дополнительно преобразует число секунд в микросекунды и добавляет значение поля микросекунд.

```
date_part  
-----  
56123459  
(1 строка)
```

Выполните эту команду, последовательно указывая в качестве первого параметра значения `microsecond`, `millisecond`, `second`, `minute`, `hour`, `day`, `week`, `month`, `year`, `decade`, `century`, `millennium`. Можно использовать сокращения этих наименований, которые приведены в предыдущем задании.

Обратите внимание, что в ряде случаев выводится не просто конкретное поле (фрагмент) из временной отметки, а некоторый продукт переработки этого поля. Например, если в качестве первого параметра функции `extract` в вышеприведенной команде указать `cent` (век), то мы получим в ответ не 19 (что и было бы буквальным значением поля «век»), а 20, поскольку 1999 год принадлежит двадцатому веку.

28. Функция `extract` может работать не только с данными типа `timestamp`, но также и с данными типа `interval`. Самостоятельно ознакомьтесь с этими возможностями по документации (см. раздел 9.9 «Операторы и функции даты/времени»).
- 29.* В тексте главы мы создавали таблицу с помощью команды

```
CREATE TABLE databases  
( is_open_source boolean,  
  dbms_name text  
);
```

и заполняли ее данными.

```
INSERT INTO databases VALUES ( TRUE, 'PostgreSQL' );  
INSERT INTO databases VALUES ( FALSE, 'Oracle' );  
INSERT INTO databases VALUES ( TRUE, 'MySQL' );  
INSERT INTO databases VALUES ( FALSE, 'MS SQL Server' );
```

Как вы думаете, являются ли все приведенные ниже команды равнозначными в смысле результатов, получаемых с их помощью?

```
SELECT * FROM databases WHERE NOT is_open_source;  
SELECT * FROM databases WHERE is_open_source <> 'yes';  
SELECT * FROM databases WHERE is_open_source <> 't';  
SELECT * FROM databases WHERE is_open_source <> '1';  
SELECT * FROM databases WHERE is_open_source <> 1;
```

- 30.* Обратимся к таблице, создаваемой с помощью команды

```
CREATE TABLE test_bool  
( a boolean,  
  b text  
);
```

Как вы думаете, какие из приведенных ниже команд содержат ошибку?

```
INSERT INTO test_bool VALUES ( TRUE, 'yes' );
INSERT INTO test_bool VALUES ( yes, 'yes' );
INSERT INTO test_bool VALUES ( 'yes', true );
INSERT INTO test_bool VALUES ( 'yes', TRUE );
INSERT INTO test_bool VALUES ( '1', 'true' );
INSERT INTO test_bool VALUES ( 1, 'true' );
INSERT INTO test_bool VALUES ( 't', 'true' );
INSERT INTO test_bool VALUES ( 't', truth );
INSERT INTO test_bool VALUES ( true, true );
INSERT INTO test_bool VALUES ( 1::boolean, 'true' );
INSERT INTO test_bool VALUES ( 111::boolean, 'true' );
```

Проверьте свои предположения практически, выполнив эти команды.

- 31.* Пусть в таблице `birthdays` хранятся даты рождения какой-то группы людей. Создайте эту таблицу с помощью команды

```
CREATE TABLE birthdays
( person text NOT NULL,
  birthday date NOT NULL );
```

Добавьте в нее несколько строк, например:

```
INSERT INTO birthdays VALUES ( 'Ken Thompson', '1955-03-23' );
INSERT INTO birthdays VALUES ( 'Ben Johnson', '1971-03-19' );
INSERT INTO birthdays VALUES ( 'Andy Gibson', '1987-08-12' );
```

Давайте выберем из таблицы `birthdays` строки для всех людей, родившихся в каком-то конкретном месяце, например, в марте:

```
SELECT * FROM birthdays
WHERE extract( 'mon' from birthday ) = 3;
```

В этой команде в вызове функции `extract` имеет место неявное приведение типов, т. к. ее вторым параметром должно быть значение типа `timestamp`. Полагаться на неявное приведение типов можно не всегда.

```
   person      | birthday
-----+-----
Ken Thompson   | 1955-03-23
Ben Johnson    | 1971-03-19
(2 строки)
```

Если нам потребуется выяснить, кто из этих людей достиг возраста, скажем, 40 лет на момент выполнения запроса, то команда может быть такой (в последнем столбце показана дата достижения возраста 40 лет):

```
SELECT *, birthday + '40 years'::interval
FROM birthdays
WHERE birthday + '40 years'::interval < current_timestamp;
```

person	birthday	?column?
Ken Thompson	1955-03-23	1995-03-23 00:00:00
Ben Johnson	1971-03-19	2011-03-19 00:00:00

(2 строки)

Можно заменить `current_timestamp` на `current_date`:

```
SELECT *, birthday + '40 years'::interval
FROM birthdays
WHERE birthday + '40 years'::interval < current_date;
```

А вот если мы захотим определить точный возраст каждого человека на текущий момент времени, то как получить этот результат?

Первый вариант таков:

```
SELECT *, ( current_date::timestamp - birthday::timestamp )::interval
FROM birthdays;
```

person	birthday	interval
Ken Thompson	1955-03-23	22477 days
Ben Johnson	1971-03-19	16637 days
Andy Gibson	1987-08-12	10647 days

(3 строки)

Этот вариант не дает результата, представленного в удобной форме: он показывает возраст в днях, а для пересчета числа дней в число лет нужны дополнительные действия. Хотя, наверное, возможны ситуации, когда требуется определить возраст именно в днях.

В PostgreSQL предусмотрена специальная функция, позволяющая решить нашу задачу простым способом. Самостоятельно найдите ее описание в документации (см. раздел 9.9 «Операторы и функции даты/времени») и напишите команду с ее использованием.

32. Изучая приемы работы с массивами, можно, как и в других случаях, пользоваться способностью команды SELECT обходиться без создания таблиц. Покажем лишь два примера.

Для объединения (конкатенации) массивов служит функция `array_cat`:

```
SELECT array_cat( ARRAY[ 1, 2, 3 ], ARRAY[ 3, 5 ] );
```

```
array_cat
-----
{1,2,3,3,5}
(1 строка)
```

Удалить из массива элементы, имеющие указанное значение, можно таким образом:

```
SELECT array_remove( ARRAY[ 1, 2, 3 ], 3 );
```

```
array_remove
-----
{1,2}
(1 строка)
```

Для работы с массивами предусмотрено много различных функций и операторов, представленных в разделе документации 9.18 «Функции и операторы для работы с массивами». Самостоятельно ознакомьтесь с ними, используя описанную технологию работы с командой SELECT.

- 33.* В разделе документации 8.15 «Массивы» сказано, что массивы могут быть многомерными и в них могут содержаться значения любых типов. Давайте сначала рассмотрим одномерные массивы *текстовых* значений.

Предположим, что пилоты авиакомпании имеют возможность высказывать свои пожелания насчет конкретных блюд, из которых должен состоять их обед во время полета. Для учета пожеланий пилотов необходимо модифицировать таблицу `pilots`, с которой мы работали в разделе 4.5.

```
CREATE TABLE pilots
( pilot_name text,
  schedule integer[],
  meal text[]
);
```

Добавим строки в таблицу:

```
INSERT INTO pilots
VALUES ( 'Ivan', '{ 1, 3, 5, 6, 7 }'::integer[],
        '{ "сосиска", "макароны", "кофе" }'::text[]
      ),
      ( 'Petr', '{ 1, 2, 5, 7 }'::integer [],
        '{ "котлета", "каша", "кофе" }'::text[]
      ),
      ( 'Pavel', '{ 2, 5 }'::integer[],
        '{ "сосиска", "каша", "кофе" }'::text[]
      ),
      ( 'Boris', '{ 3, 5, 6 }'::integer[],
        '{ "котлета", "каша", "чай" }'::text[]
      );
```

INSERT 0 4

Обратите внимание, что каждое из текстовых значений, включаемых в литерал массива, заключается в двойные кавычки, а в качестве типа данных указывается `text[]`.

Вот что получилось:

```
SELECT * FROM pilots;
```

pilot_name	schedule	meal
Ivan	{1,3,5,6,7}	{сосиска,макароны,кофе}
Petr	{1,2,5,7}	{котлета,каша,кофе}
Pavel	{2,5}	{сосиска,каша,кофе}
Boris	{3,5,6}	{котлета,каша,чай}

(4 строки)

Давайте получим список пилотов, предпочитающих на обед сосиски:

```
SELECT * FROM pilots WHERE meal[ 1 ] = 'сосиска';
```

pilot_name	schedule	meal
Ivan	{1,3,5,6,7}	{сосиска,макароны,кофе}
Pavel	{2,5}	{сосиска,каша,кофе}

(2 строки)

Предположим, что руководство авиакомпании решило, что пища пилотов должна быть разнообразной. Оно позволило им выбрать свой рацион на каждый из четырех дней недели, в которые пилоты совершают полеты. Для нас это решение руководства выливается в необходимость модифицировать таблицу, а именно: столбец `meal` теперь будет содержать двумерные массивы. Определение этого столбца станет таким: `meal text[][]`.

Задание. Создайте новую версию таблицы и соответственно измените команду `INSERT`, чтобы в ней содержались литералы *двумерных* массивов. Они будут выглядеть примерно так:

```
'{ { "сосиска", "макароны", "кофе" },
  { "котлета", "каша", "кофе" },
  { "сосиска", "каша", "кофе" },
  { "котлета", "каша", "чай" } }':text[][]
```

Сделайте ряд выборов и обновлений строк в этой таблице. Для обращения к элементам двумерного массива нужно использовать два индекса. Не забывайте, что по умолчанию номера индексов начинаются с единицы.

34. В тексте раздела 4.6 мы выполняли обновление JSON-объекта с помощью функции `jsonb_set`: добавляли значение в массив. Для обновления скалярных значений, например, по ключу `trips`, можно сделать так:

```
UPDATE pilot_hobbies
SET hobbies = jsonb_set( hobbies, '{ trips }', '10' )
WHERE pilot_name = 'Pavel';
```

UPDATE 1

Второй параметр функции — это путь в пределах JSON-объекта. Он теперь представляет собой лишь имя ключа. Однако его необходимо заключить в фигурные скобки. Третий параметр — это новое значение. Хотя оно числовое, но все равно требуется записать его в одинарных кавычках.

```
SELECT pilot_name, hobbies->'trips' AS trips FROM pilot_hobbies;
```

```
pilot_name | trips
-----+-----
Ivan       | 3
Petr       | 2
Boris      | 0
Pavel      | 10
(4 строки)
```

Задание. Самостоятельно выполните изменение значения по ключу `home_lib` в одной из строк таблицы.

35. Изучая приемы работы с типами JSON, можно, как и в случае с массивами, пользоваться способностью команды `SELECT` обходиться без создания таблиц.

Покажем лишь один пример. Добавить новый ключ и соответствующее ему значения в уже существующий объект можно оператором `||`:

```
SELECT '{ "sports": "хоккей" }'::jsonb || '{ "trips": 5 }'::jsonb;
```

```
-----
?column?
```

```
-----
{"trips": 5, "sports": "хоккей"}
(1 строка)
```

Для работы с типами JSON предусмотрено много различных функций и операторов, представленных в разделе документации 9.15 «Функции и операторы JSON». Самостоятельно ознакомьтесь с ними, используя описанную технологию работы с командой `SELECT`.

- 36.* Объекты JSON в разных строках таблицы могут иметь различные наборы ключей. Добавьте дополнительный ключ и соответствующее ему значение в JSON-объект какой-нибудь строки таблицы `pilots`. Используйте оператор `||`.
37. Объекты JSON позволяют не только добавлять в них новые ключи, но также и удалять из них ключи существующие. Удалите один из ключей из JSON-объекта какой-нибудь строки таблицы `pilots`. Соответствующее ему значение будет также удалено, т. к. без ключа оно не может существовать. Воспользуйтесь оператором `-`.

